혼이음

# SW개발/HW제작 설계서

## 프로젝트 명 : 주차장 공간이 협소한 빌라를 위한 인공지능 관리 어플리케이션 “스마트 파킹”

2024. 07. 16

(팀명) 주차의 달인

# 수행 단계별 주요 산출물

| 단계 | 산출물 | 일반<br>·모바일 APP<br>·Web 등 | 응용 소프트웨어<br>·빅데이터<br>·인공지능<br>·블록체인 등 | 응용 하드웨어<br>·IoT<br>·로봇<br>·드론 등 |
|---|---|---|---|---|
| 환경 분석 | 시장/기술 환경 분석서 | △ | △ | △ |
| | 설문조사 결과서 | △ | △ | △ |
| | 인터뷰 결과서 | △ | △ | △ |
| 요구사항 분석 | 요구사항 정의서 | ○ | ○ | ○ |
| | 유스케이스 정의서 | ▲ | ▲ | ▲ |
| 아키텍처 설계 | 서비스 구성도(시스템 구성도) | ○ | ○ | ○ |
| | 서비스 흐름도(데이터 흐름도) | △ | ○ | △ |
| | UI/UX 정의서 | △ | △ | △ |
| | 하드웨어/센서 구성도 | - | - | ○ |
| 기능 설계 | 메뉴 구성도 | ○ | ○ | ○ |
| | 화면 설계서 | ○ | ○ | △ |
| | 엔티티 관계도 | ○ | ○ | △ |
| | 기능 처리도(기능 흐름도) | ○ | ○ | ○ |
| | 알고리즘 명세서/설명서 | △ | ○ | ○ |
| | 데이터 수집처리 정의서 | - | ○ | - |
| | 하드웨어 설계도 | - | - | ○ |
| 개발 / 구현 | 프로그램 목록 | ○ | ○ | ○ |
| | 테이블 정의서 | ○ | ○ | △ |
| | 핵심 소스코드 | ○ | ○ | ○ |

※ ○ 필수, △ 선택

# 1. 커져가는 주차 문제

**살인마 만드는 '주차 전쟁'…"알아서 해결해라" 정부는 나몰라라**

전병육 기자 cinema@mk.co.kr

입력 : 2023-11-03 19:24:31

가

늘어난 자동차수에 비해
충분하지 않은 주차공간
주차관련 민원 빗발쳐도
사적 자치영역이라며 '난색'




서울 강남구 압구정초등학교 인근에서 강남구청 주차단속차량이 이동하고 있다. [사진 출처=연합뉴스]

주차 갈등은 하루이틀 일이 아니지만 최근 그 양상이 극에 달하고 있다. 3일 국민권익위원회에 따르면, 올해 1~9월 국민신문고에 접수된 '사유지 불법주차' 민원 건수는 6만1265건을 기록했다. 역대 최고치였던 작년 한 해 동안의 민원 건수(5만4778건)를 올해 1~3분기 만에 넘어 신기록을 경신했다. 사유지 불법주차 민원은 2020년 2만4817건, 2021년 4만44434건, 2022년 5만4778건으로 가파르게 상승 중이다.

불법주차 민원 건도 매년 급증하고 있다. 2020년 처음 300만건을 돌파한 이후 2021년 340만건, 2022년 353만건을 기록했다. 올해 1~9월은 319만건으로 이 부문 민원 건수 역시 역대 최고치를 기록할 것으로 전망된다.

주차난이 심화되는 건 주차 공간에 비해 차량 수가 많기 때문이다. 지난해 자동차 등록대 수는 2550만대에 달했다. 2013년 1940만대였으나 10년 사이에 31% 증가했다. 하지만 구축 아파트와 다가구·다세대 주택의 주차장 확보율은 낮은 편이다. 자동차 보유가 대중화가 되기 전에 주차 공간을 충분히 확보하지 않았기 때문이다. 현행법상 사유지에 대한 주차 질서 위반에 대해서는 과태료 부과, 강제 이동 등의 행정조치가 어려운 상태다.

현재 세대당 차량 수가 아파트 같은 경우 1.1~1.6 정도의 통계를 가지고 있음.
빌라(다세대 주택) 경우에는 평균 1대 이하로 계속해서 늘어나는 차량 수에 비해 주차장 수가 현저히 부족함.

# 2. AI와 영상분석 시장

## 연평균 33% 성장하는 글로벌 AI영상분석 시장...도전하는 K-스타트업들 '주목'

전 세계 AI영상분석 시장, 2023년 181억불→2028년 753억불
메이아이, CCTV 영상처리 AI솔루션 '메쉬', 60억원 투자유치
써모아이, 열화상 카메라 분석용 AI 솔루션, 9억원 프리A 유치
클레(AI 3차원 머신비전)-트윌브랩스(영상이해 초거대 AI)도 활약

[더스탁=김동진 기자] 인공지능(AI)을 활용한 영상분석 시장이 급성장하고 있다.

글로벌 시장조사업체 '모도인텔리전스'에 따르면 전 세계 AI 영상분석 시장규모는 2023년 181억1000만달러 수준에서 연평균 33% 성장해 오는 2028년엔 753억5000만달러(약 99조원)의 거대시장이 될 전망이다.

출처 : 더스탁(The Stock)(http://www.the-stock.kr)



출처: 픽사베이

# 2. AI와 영상분석 시장




(사진=마켓앤마켓)

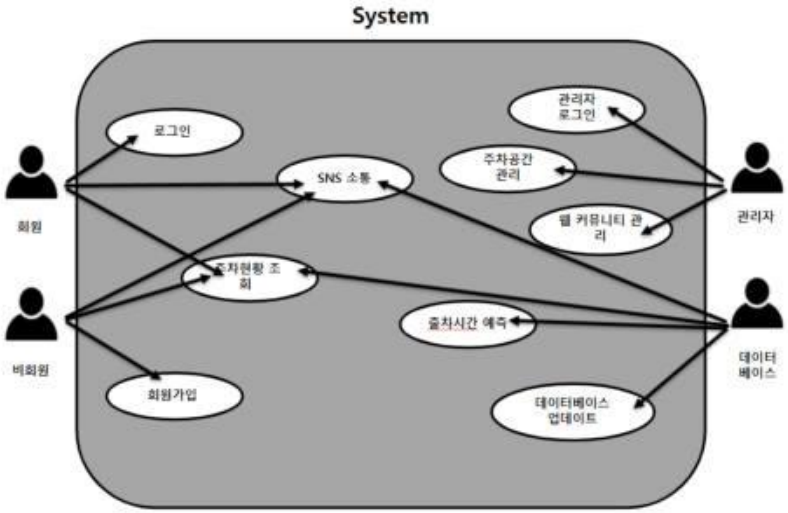
"2030년 AI 글로벌 시장 규모 1800조...2023년 9배 달할 것"

# 전체 리스트 전부 한글 파일에서 적은것도 모든 시나리오 포함 노란색 수정 바람(알림 기능 구현 안 함)

| 구분 | 기능 | 설명 |
|---|---|---|
| S/W | 실시간 빌라 주차 현황판 검색 | 웹캠을 통해 실시간 주차 현황을 확인할 수 있음. |
| | SNS, 게시판 | 주민들 간의 필요한 정보를 올리고 소통할 수 있음. |
| | 영상처리 | 웹캠으로 받아온 주차 현황과 차량 번호판을 영상처리를 통해 처리하고 라벨링할 수 있음. |
| | 예상 출차 시간 제공 | 이전에 쌓인 각 차량의 주차 정보를 통해 예상 출차 시간을 제공 할 수 있음. |
| | 데이터 분석 | 주차 데이터를 분석하여 관리자가 주차 정책을 개선할 수 있는 인사이트 제공함. |
| | 사용자 인터페이스 | HTML, CSS, JavaScript를 사용하여 직관적이고 반응형 인터페이스를 제공함. |

| 구분 | 기능 | 설명 |
|---|---|---|
| H/W | 웹캠을 통한 차량 번호판 인식 | 웹캠을 이용하여 실시간으로 차량 번호판을 인식함. |
| | 모형 차량 | 모형 차량을 이용하여 빌라 주차 상황을 연출함. |
| | 차량 번호판 | 각 차량마다 다른 번호판을 통해 인식함. |
| | 빌라 주차장 | 지정된 빌라 주차장 공간을 통해 실시간 주차 현황을 제공함. |
| | 모형 빌라 주차장 | 지정된 빌라 주차장 공간을 대신하여 데모로 테스트하기 위해 만들어진 주차장. |

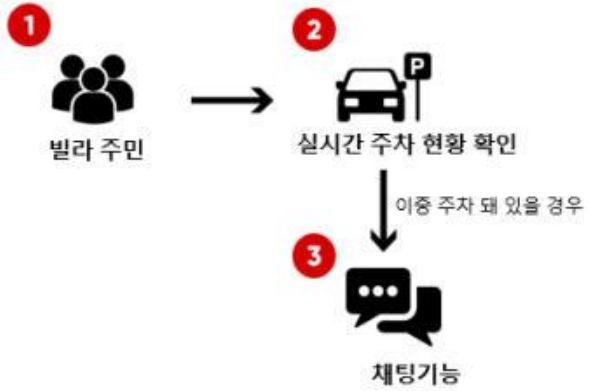
# | 유스케이스




| 항목 | 설명 | |
|---|---|---|
| 개요 | 사용자가 웹 페이지에서 실시간 주차 현황과 예측된 출차 시간을 확인할 수 있다. | |
| 관련 액터 | 주액터 | 사용자 |
| | 보조액터 | 관리자, 데이터베이스 |
| 난이도 | 실시간으로 주차 현황을 파악하고 출차 시간을 예측할 수 있는 기술 구현 능력 | |
| 선행 조건 | 사용자가 웹 페이지에 접속할 수 있는 능력 | |
| 후행 조건 | cam이 서버와 연결되어있어야 하고, WIFI 또는 데이터가 지속적으로 연결되어 있어야 한다. | |
| 시나리오 | 기본 시나리오 | 1. 사용자는 웹 페이지에 접속한다.<br>2. 웹 페이지는 실시간 주차 현황과 출차 시간 예측 정보를 사용자에게 제공한다.<br>3. 사용자는 주차 현황과 출차 시간 예측 정보를 확인한다.<br>4. 사용자는 필요에 따라 SNS를 통해 주차 공간을 예약하거나 문의한다. |
| | 대안 시나리오 | A1: 웹 페이지에 접속할 수 없을 때<br>1. 사용자는 오류 메시지를 확인한다.<br>2. 사용자는 관리자에게 연락하여 문제를 해결한다.<br>A2: 주차 공간이 모두 차있을 때<br>1. 사용자는 SNS로 소통하거나 다른 주차 공간을 찾는다. |

# | 서비스 구성도 - 서비스 시나리오

## 주차 돼 있는 차를 나가야 할 때




## 주차를 하러 가야 할 때



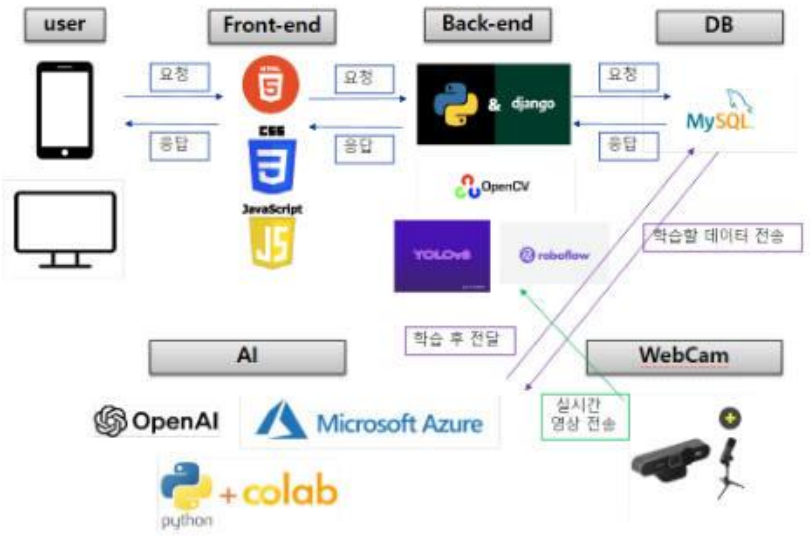

**주차 돼 있는 차를 나가야 할 때**

1. 빌라 주민이 웹페이지에 접속함.
2. 실시간 주차 현황을 클릭해 확인함.
3. 이중 주차가 있을 시에 미리 채팅을 통해 연락함.

**주차를 하러 가야 할 때**

1. 빌라 주민이 웹페이지에 접속함.
2. 실시간 주차 현황을 클릭해 확인함.
3. 미리 확인 후에 주차 할 곳이 없을 경우 다른 주차장을 이용함.

# | 서비스 구성도 - 서비스 시나리오




## Front-end

**기능:**

1. 사용자가 주차 현황, 출차 시간 예측 등의 데이터를 조회할 수 있도록 요청을 송신.
2. HTML, CSS, JavaScript를 이용해 사용자 인터페이스를 구성.
3. 사용자의 요청을 Back-end로 전달하고, Back-end에서 전달받은 데이터를 사용자에게 표시.

**흐름:**

1. 사용자(User)가 주차 현황 또는 출차 시간 예측 데이터를 요청.
2. 요청을 받아 Front-end가 Back-end로 데이터를 요청.
3. Back-end에서 데이터를 전달받아 사용자에게 표시.

## Back-end

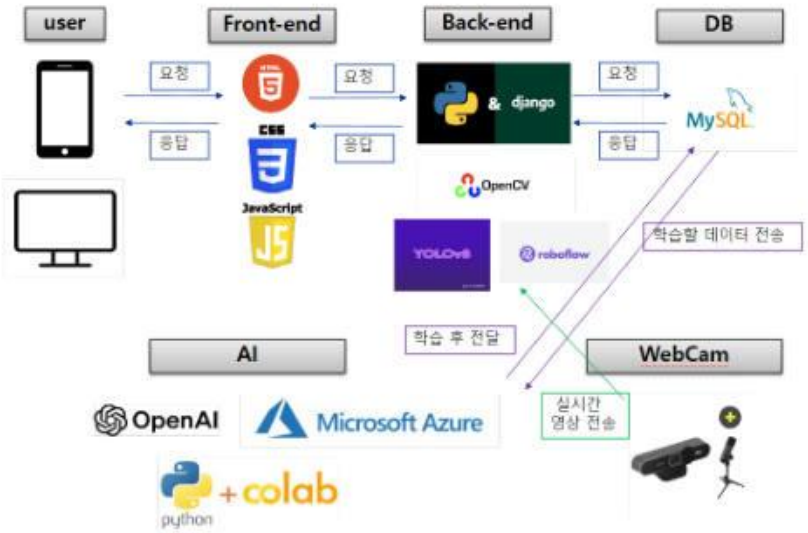
**기능:**

1. Front-end로부터 받은 요청에 따라 데이터베이스(DB)와 상호작용.
2. OpenCV와 Tesseract OCR을 이용해 주차 현황 데이터를 처리.
3. Django 프레임워크를 사용하여 API를 제공.
4. AI 모델을 통해 학습된 데이터를 바탕으로 예측 결과를 제공.

**흐름:**

1. Front-end로부터 데이터를 요청받음.
2. 요청에 따라 데이터베이스(DB)로부터 필요한 데이터를 요청.
3. OpenCV와 Tesseract OCR을 이용해 실시간 영상 데이터를 처리.
4. AI 모델을 통해 출차 시간을 예측.
5. 예측된 데이터를 Front-end로 전달.

# | 서비스 구성도 - 서비스 시나리오




**DB**

**기능:**

1. 주차 현황, 출차 시간, 사용자 정보 등의 데이터를 저장.
2. Back-end로부터 데이터 요청이 오면 알맞은 데이터를 전달.

**흐름:**

1. Back-end로부터 데이터 요청을 받음.
2. 저장되어 있는 알맞은 데이터를 Back-end로 전달.
3. AI가 주차 현황 데이터를 학습할 수 있도록 데이터를 전달.
4. AI로부터 요약된 데이터를 저장.

**AI**

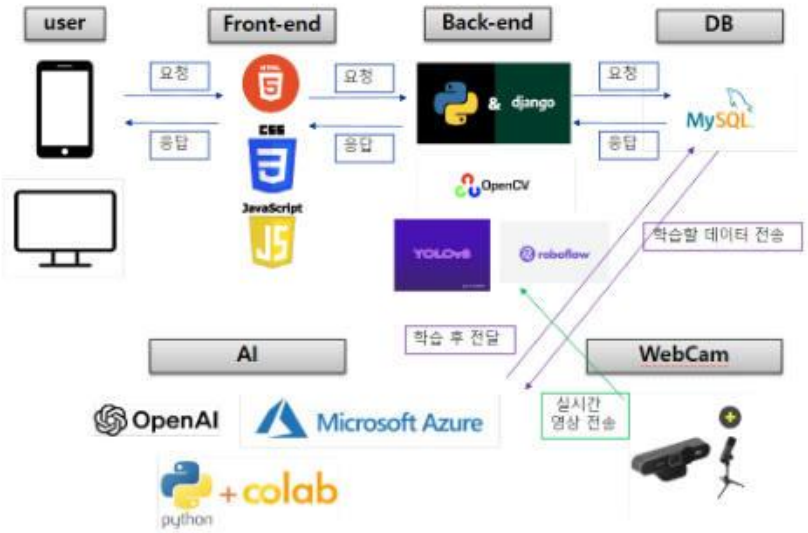
**기능:**

1. DB로부터 주차 현황 데이터를 받아 딥러닝 모델을 통해 학습.
2. 출차 시간을 예측하여 결과를 DB에 저장.
3. Google Colab과 같은 GPU 서버를 이용해 학습을 수행.

**흐름:**

1. DB로부터 주차 현황 데이터를 수신.
2. 딥러닝 모델을 이용해 데이터를 학습.
3. 학습된 모델을 이용해 출차 시간을 예측.
4. 예측 결과를 DB에 저장.

# | 서비스 구성도 - 서비스 시나리오




## Server

**기능:**

1. Microsoft Azure를 이용하여 데이터베이스 서버와 배포 서버로 사용됨
   .
2. 웹 서비스가 안정적으로 동작할 수 있도록 서버 환경을 제공.

**흐름:**

1. Azure 서버에서 Back-end와 DB가 동작.
2. Google Colab에서 AI 학습을 위한 GPU 서버를 제공.
3. 서버에서 실시간으로 주차 현황 데이터를 수집하고 전송.

## WebCam

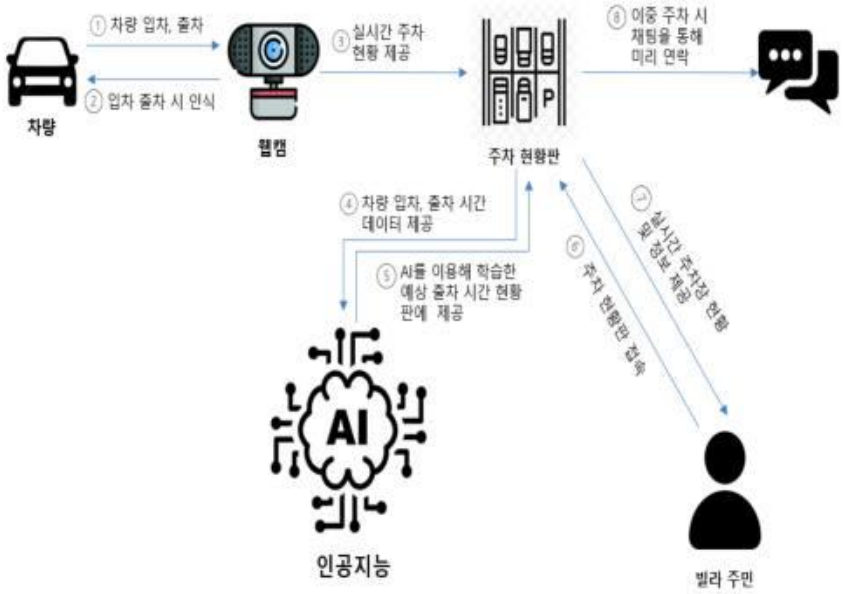
**기능:**

1. 실시간 주차 현황 영상을 제공하여 데이터 수집에 사용.

**흐름:**

1. WebCam을 통해 실시간으로 주차 현황 영상을 전송.
2. 전송된 영상을 OpenCV와 Tesseract OCR을 이용해 처리.
3. 처리된 데이터를 AI가 학습하고 예측 결과를 도출.

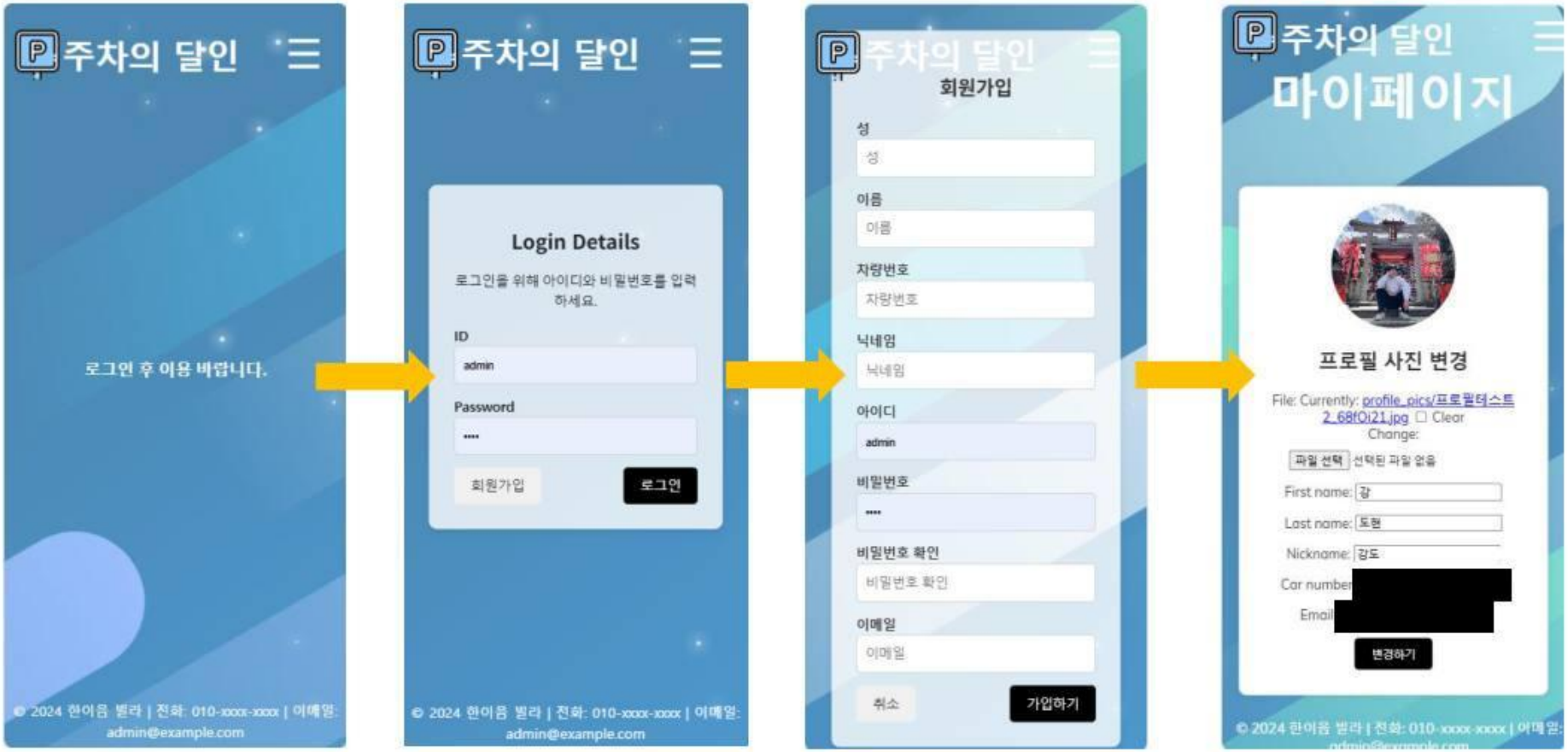
# | 서비스 흐름도




1 .차량 입차 및 출차.
2. 웹캠을 통해 차량의 입차 및 출차 인식.
3. 주차 현황판에 실시간 주차 현황을 제공.
4. AI로부터 차량의 입차, 출차 데이터 제공.
5. 학습한 데이터를 통해 주차 차량별 예상 출차 시간을 제공.
6. 빌라 주민이 주차 현황판에 접속.
7. 실시간 주차 현황을 빌라 주민에게 제공.
8. 만약 이중 주차시에 채팅을 통해 미리 이중 주차 차주에게 연락하여 시간을 절약 할 수 있음.

# | UI/UX 정의서 - 화면 설계서

- 로그인 전 홈페이지 > 회원가입 신청 > 로그인 > 마이페이지



# | UI/UX 정의서 - 화면 설계서

- 홈페이지 > 각 기능 (주차현황, 예상출차시간, 게시판) > 각페이지 > 추가 정보 입력











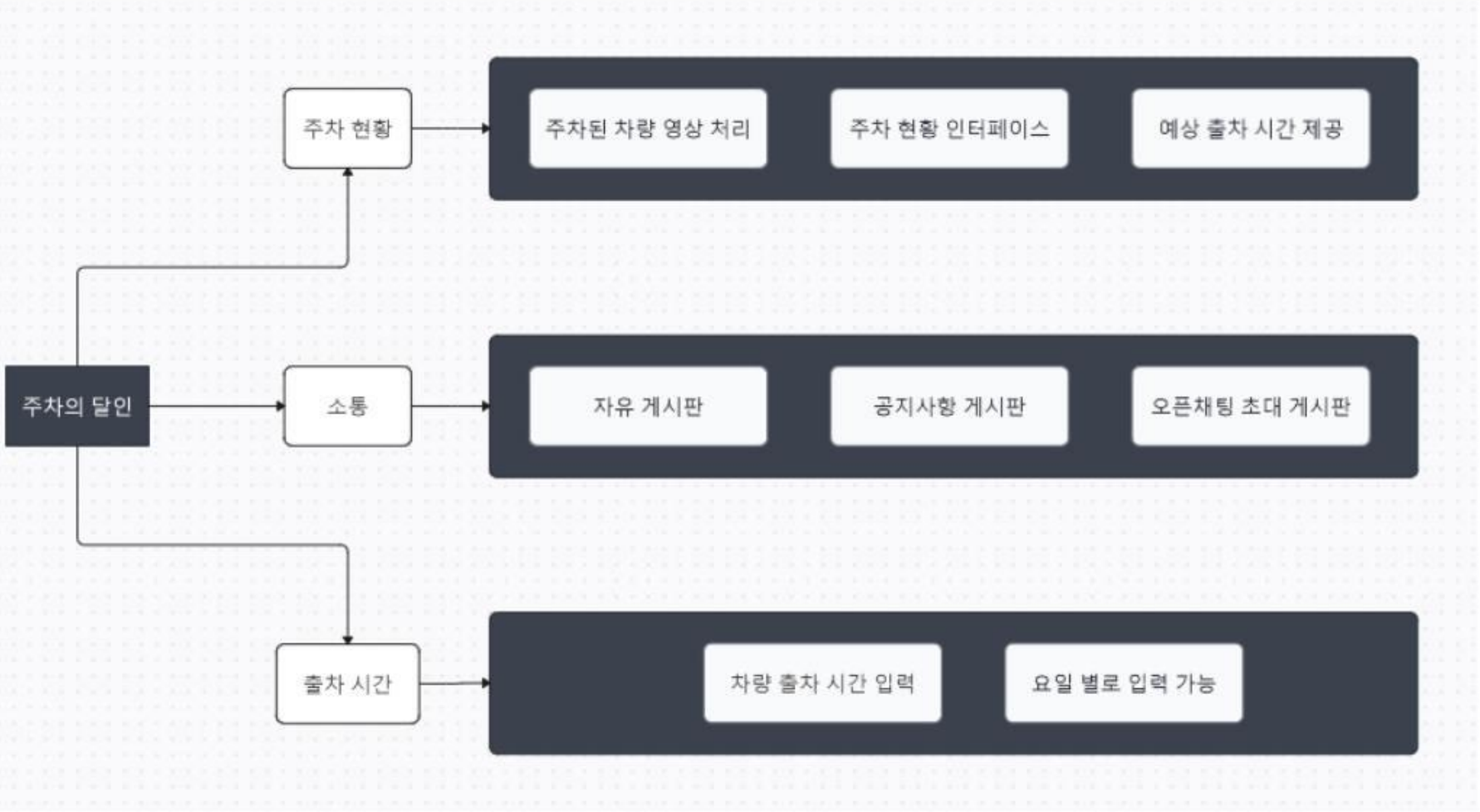




## | 메뉴 구성도


<메뉴 구성도>
스플래쉬 화면
회원가입
로그인
홈페이지
네비게이션
홈
홈페이지
주변주차장
google map api
마이페이지
프로필 정보 수정
차량번호
이메일 주소
닉네임
로그아웃
스플래쉬 화면
홈페이지 중단
주차현황
주차 현황 인터페이스
출차시간
출차시간 입력 인터페이스
게시판
자유 게시판
오픈채팅 초대
공지사항

# | 메뉴 구성도


주차의 달인
주차 현황
주차된 차량 영상 처리
주차 현황 인터페이스
예상 출차 시간 제공
소통
자유 게시판
공지사항 게시판
오픈채팅 초대 게시판
출차 시간
차량 출차 시간 입력
요일 별로 입력 가능

# | 화면 설계서




| | |
|---|---|
| 기능 번호 | LOG-01-01 |
| 기능 명 | 로그인 전 화면 |
| 기능설명 | 홈페이지의 첫 화면, 네비게이션 바를 통해서 로그인창과 아이디/비밀번호 찾기, 그리고 회원가입 창으로 넘어갈 수 있음. |
| 처리내용 | ■기존회원의 로그인<br>자신이 가입했던 계정으로 로그인을 할 수 있음.<br>네비게이션의 로그인 버튼을 누르면 로그인 창으로 넘어감.<br>■기존 회원의 아이디/비밀번호 찾기<br>자신이 가입했던 계정을 찾을 수 있음.<br>네비게이션의 '아이디/비밀번호 찾기' 버튼을 눌러 신규 회원의 회원가입 창으로 넘어감.<br>■신규 회원의 회원가입<br>본인의 정보와 이메일을 입력하고 회원가입을 할 수 있음.<br>네비게이션의 '회원가입' 버튼을 눌러 신규 회원의 회원가입 창으로 넘어감<br>. |
| 비고 | ■구현 기능 :<br>네비게이션 바( 홈, 로그인, 회원가입, 아이디/비밀번호 찾기 ) |
| 요구사항 명 | 홈페이지-로그오프 |

# | 화면 설계서



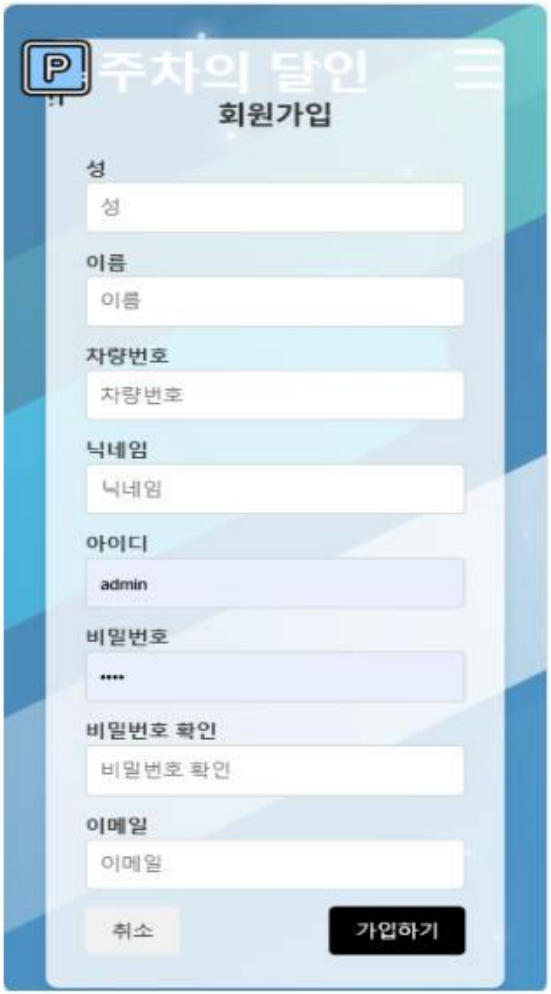

| | |
|---|---|
| 기능 번호 | LOG-01-02 |
| 기능 명 | 로그인 |
| 기능설명 | 로그인 화면,<br>기존회원은 자신이 가입했던 계정으로 로그인을 할 수 있음. |
| 처리내용 | ■네비게이션 바<br>네비게이션 바로, Home, 회원가입, 아이디/비밀번호 찾기 창으로 이동할 수 있음.<br>■기존회원의 로그인<br>자신이 가입했던 계정으로 로그인을 할 수 있음. |
| 비고 | ■구현 기능 :<br>로그인, 네비게이션 바(홈, 회원가입, 아이디/비밀번호 찾기) |
| 요구사항 명 | 로그인 |

# | 화면 설계서




| | |
|---|---|
| 기능 번호 | LOG-01-03 |
| 기능 명 | 아이디/비밀번호 찾기 |
| 기능설명 | 아이디/비밀번호 찾기 화면,<br>기존 회원은 가입한 계정을 찾을 수 있음. |
| 처리내용 | ■네비게이션 바<br>네비게이션 바로, Home, 회원가입, 로그인 창으로 이동할 수 있음.<br>■기존 회원의 아이디/비밀번호 찾기<br>본인의 정보와 이메일을 입력하면, 메일로 인증하여 아이디 찾기와 비밀번호 재설정을 할 수 있음. |
| 비고 | ■구현 기능 :<br>메일과 본인 정보를 통해 아이디/비밀번호 찾기, 네비게이션 바(홈, 회원가입, 로그인) |
| 요구사항 명 | 아이디/비밀번호 찾기 |

# | 화면 설계서




| | |
|---|---|
| 기능 번호 | LOG-01-04 |
| 기능 명 | 회원가입 |
| 기능설명 | 회원가입 화면,<br>신규 회원은 회원가입을 할 수 있음. |
| 처리내용 | ■네비게이션 바<br>네비게이션 바로, Home, 로그인, 아이디/비밀번호 찾기 창으로 이동할 수 있음.<br>■신규 회원의 회원가입<br>본인의 정보와 이메일을 입력하고 회원가입을 할 수 있음. |
| 비고 | ■구현 기능 :<br>회원가입, 네비게이션 바 |
| 요구사항 명 | 회원가입 |

# | 화면 설계서

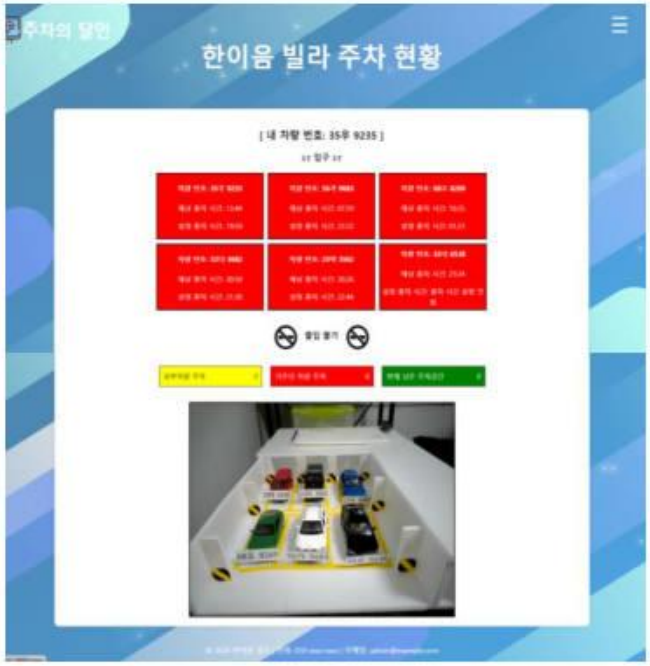






| | |
|---|---|
| 기능 번호 | HOME-01-01 |
| 기능 명 | 홈 화면 |
| 기능설명 | 홈페이지의 첫 화면,<br>주차 현황, 출차 시간 설정, 게시판 창으로 이동 할 수 있음. |
| 처리내용 | ■주차 현황<br>주차 현황 페이지로 넘어감.<br>■출차 시간 설정<br>출차 시간 설정 페이지로 넘어감.<br>■게시판<br>게시판 목록 페이지로 넘어감. |
| 비고 | ■구현 기능 :<br>각 페이지로 이동 |
| 요구사항 명 | 홈페이지-로그인 |

# | 화면 설계서




| | |
|---|---|
| 기능 번호 | HOME-01-02 |
| 기능 명 | 주차 현황 페이지 |
| 기능설명 | 현 주차 현황과 딥러닝을 이용하여 예상 출차 시간을 확인 할 수 있음.<br>빈 주차공간 : 초록색 , 거주인 주차: 빨간색, 외부 차량 주차 : 노란색 ,<br>빈 주차 공간 중 유저에게 추천하는 주차 공간 : 파란색 |
| 처리내용 | ■주차 현황<br>현재 주차 현황과 주차된 차량들의 예상 출차 시간을 딥러닝 강화학습으로 처리함.<br>■주차장 UI 색상 의미<br>빈 주차 공간 : 초록색<br>거주인 주차 : 빨간색<br>외부 차량 주차 : 노란색<br>빈 주차 공간 중 유저에게 추천하는 주차 공간 : 파란색 |
| 비고 | ■구현 기능 :<br>주차 현황 인터페이스, 차량 예상 출차 시간(강화 학습) |
| 요구사항 명 | 주차 현황 |

# | 화면 설계서




| | |
|---|---|
| 기능 번호 | HOME-01-03 |
| 기능 명 | 출차 시간 설정 페이지 |
| 기능설명 | 예상 출차 시간을 요일에 맞춰서 설정할 수 있음. |
| 처리내용 | ■출차 시간 설정<br>예상 출차 시간을 설정 할 수 있음.<br><br>■요일 설정<br>요일을 설정할 수 있음. |
| 비고 | ■구현 기능 :<br>예상 출차 시간 설정 기능 |
| 요구사항 명 | 출차시간 설정 |

# | 화면 설계서



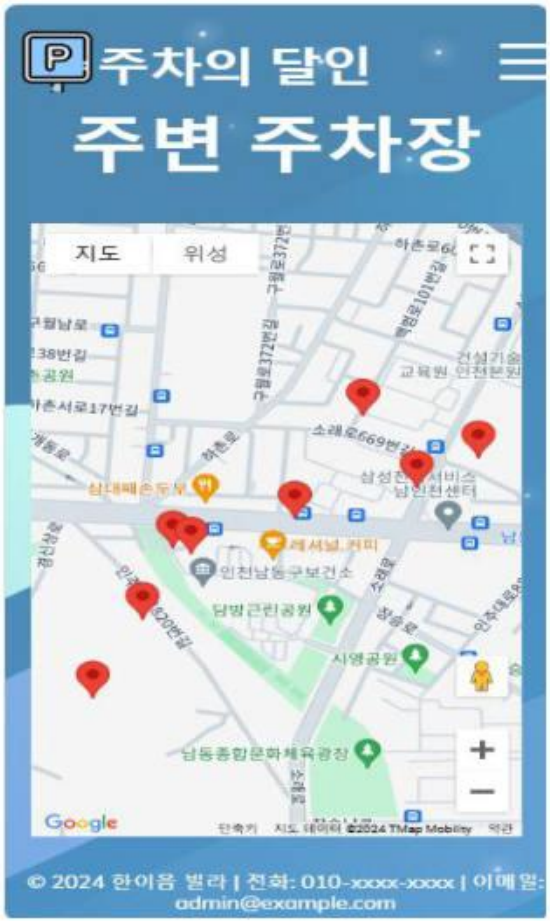

| | |
|---|---|
| 기능 번호 | HOME-01-04 |
| 기능 명 | 게시판 목록 페이지 |
| 기능설명 | 게시판 목록 화면,<br>주차 현황, 출차 시간 설정, 게시판 창으로 이동 할 수 있음. |
| 처리내용 | ■주차 현황<br>주차 현황 페이지로 넘어감.<br>■출차 시간 설정<br>출차 시간 설정 페이지로 넘어감.<br>■게시판<br>게시판 목록 페이지로 넘어감. |
| 비고 | ■구현 기능 :<br>각 페이지로 이동 |
| 요구사항 명 | 게시판 목록 |

# | 화면 설계서




| | |
|---|---|
| 기능 번호 | HOME-01-04-01 |
| 기능 명 | 자유게시판 페이지 |
| 기능설명 | 자유롭게 아무 정보나 쓸 수 있는 페이지 |
| 처리내용 | ■자유 게시판<br>게시판 목록, 게시글 작성, 해당 게시글로 이동, 게시글 검색 기능을 수행함. |
| 비고 | ■구현 기능 :<br>게시판 목록, 게시글 작성, 해당 게시글로 이동, 게시글 검색 |
| 요구사항 명 | 자유게시판 목록 |

# | 화면 설계서




| | |
|---|---|
| 기능 번호 | HOME-01-05 |
| 기능 명 | 주변 주차장 현황 |
| 기능설명 | 주변 주차장 위치 제공 |
| 처리내용 | ■주변 주차장 제공<br>주변 주차장 위치를 Google map api를 통해서 제공. |
| 비고 | ■구현 기능 : 주변 주차장 위치 제공 |
| 요구사항 명 | 주변 주차장 위치 제공 |

# | 화면 설계서




| | |
|---|---|
| 기능 번호 | HOME-01-06 |
| 기능 명 | 프로필 설정 |
| 기능설명 | 자신의 프로필 정보를 입력 및 저장 |
| 처리내용 | ■프로필 사진 업데이트<br>프로필 사진 측 DB를 auth_user와 1:1릴레이션으로 묶어서 업데이트 할 수 있음. |
| 비고 | ■구현 기능 : 프로필 정보를 관리자에게 제공 |
| 요구사항 명 | 프로필 업데이트 |

# | 화면설계서 - 사용자 인터페이스(sw)

## 회원가입




## 로그인




## 메인 페이지




## 한이음 빌라 주차 현황



# | 화면설계서 - 사용자 인터페이스(sw)

## 메인 페이지




## 출차 시간 설정



# | 화면설계서 - 사용자 인터페이스(sw)

## 메인 페이지




## 게시판 목록




## 자유게시판




## 공지사항

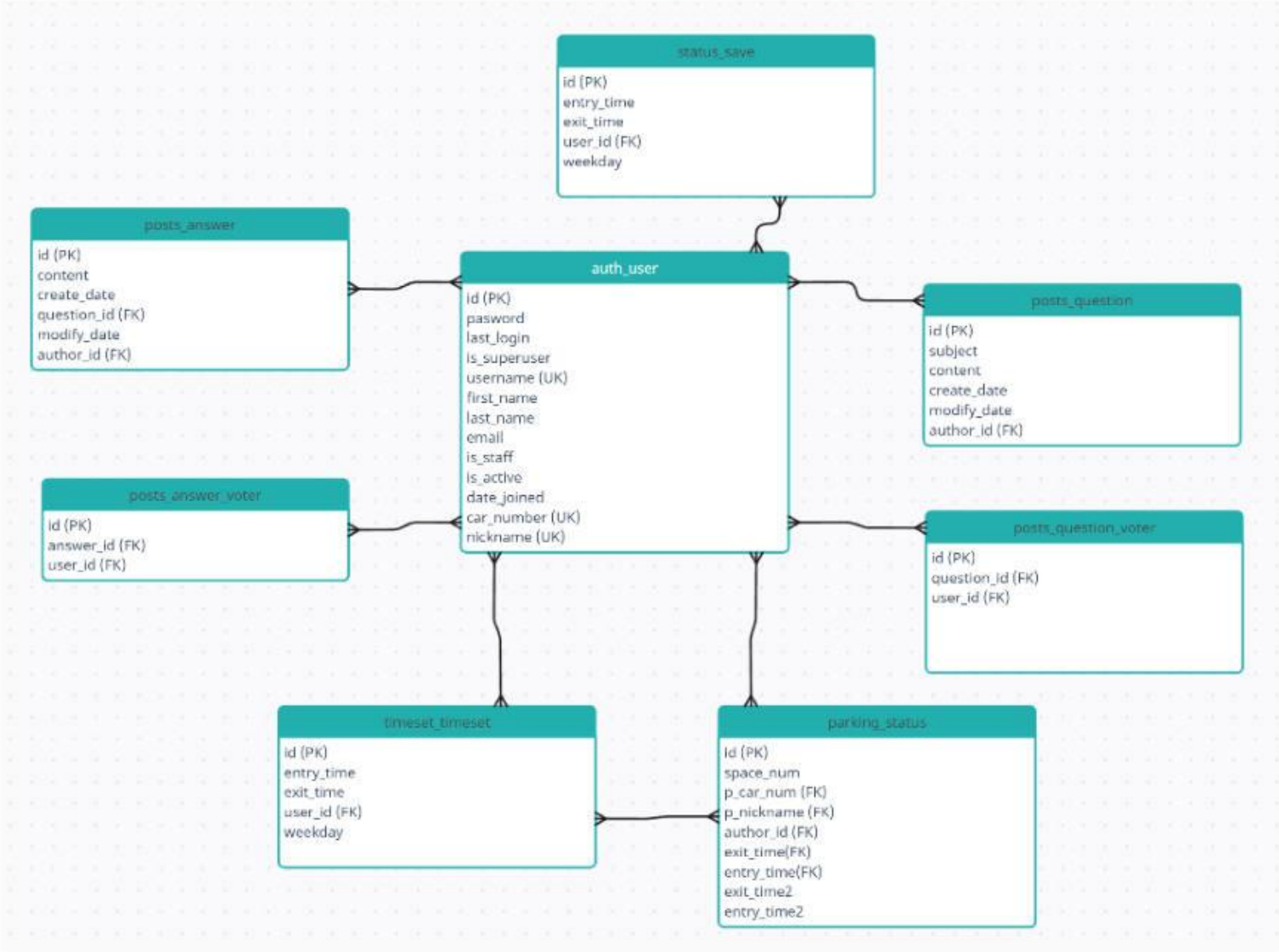



## 오픈채팅 초대

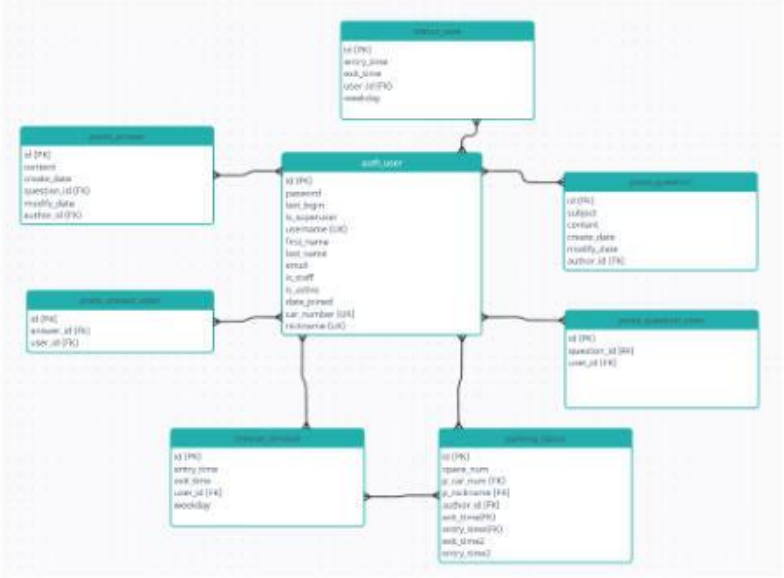


# | 개체관계도 - ERD

추가된 DB 포함해서 새로 그려야함.



# | 테이블 정의서 - ERD



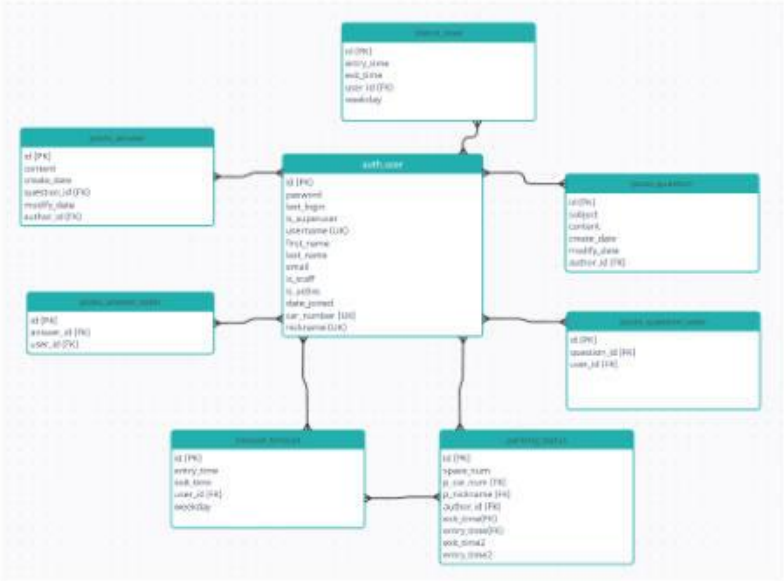
테이블 이름: auth_user

| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 사용자 고유 ID |
| password | varchar(128) | NOT NULL | 사용자 비밀번호 |
| last_login | datetime | NULL | 마지막 로그인 시간 |
| is_superuser | bool | NOT NULL | 슈퍼유저 여부 |
| username | varchar(150) | UNIQUE, NOT NULL | 사용자 이름 |
| first_name | varchar(150) | NULL | 사용자 이름 |
| last_name | varchar(150) | NULL | 사용자 성 |
| email | varchar(254) | NULL | 이메일 주소 |
| is_staff | bool | NOT NULL | 스태프 여부 |
| is_active | bool | NOT NULL | 활성화 여부 |
| date_joined | datetime | NOT NULL | 가입일 |
| car_number | varchar(150) | UNIQUE, NULL | 차량차 번호 |
| nickname | varchar(150) | UNIQUE, NULL | 사용자 닉네임 |

테이블 이름: parking_status

| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 고유 ID |
| space_num | integer | NOT NULL | 주차 공간 번호 |
| p_car_num | varchar(150) | NOT NULL | 주차된 차량 번호 |
| p_nickname | varchar(150) | NOT NULL | 주차된 차량 닉네임 |
| author_id | integer | NOT NULL | 작성자 ID |
| FOREIGN KEY (author_id) REFERENCES auth_user(id) | | | |
| FOREIGN KEY (p_nickname) REFERENCES auth_user(nickname) | | | |
| FOREIGN KEY (p_car_num) REFERENCES auth_user(car_number) | | | |

# | 테이블 정의서 - ERD



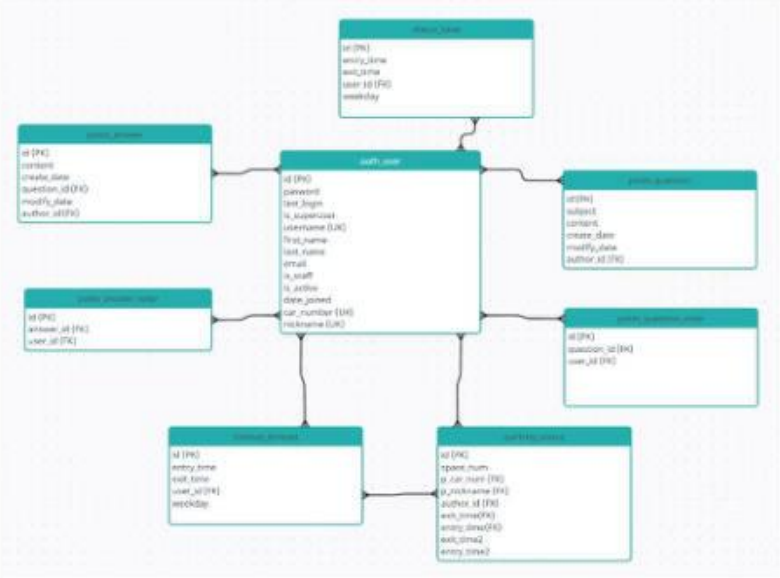
테이블 이름: posts_answer

| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 고유 ID |
| content | text | NOT NULL | 답변 내용 |
| create_date | datetime | NOT NULL | 작성일 |
| question_id | bigint | NOT NULL | 질문 ID |
| modify_date | datetime | NULL | 수정일 |
| author_id | integer | NOT NULL | 작성자 ID |
| FOREIGN KEY (question_id) REFERENCES posts_question(id) | | | |
| FOREIGN KEY (author_id) REFERENCES auth_user(id) | | | |

테이블 이름: posts_answer_voter

| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 고유 ID |
| answer_id | bigint | NOT NULL | 답변 ID |
| user_id | integer | NOT NULL | 사용자 ID |
| FOREIGN KEY (answer_id) REFERENCES posts_answer(id) | | | |
| FOREIGN KEY (user_id) REFERENCES auth_user(id) | | | |

# | 테이블 정의서 - ERD



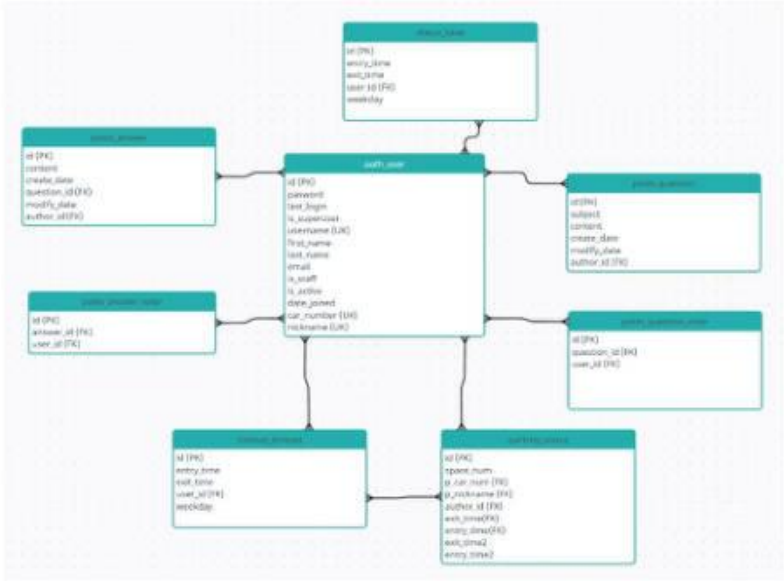
테이블 이름: posts_question

| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 고유 ID |
| subject | varchar(200) | NOT NULL | 질문 제목 |
| content | text | NOT NULL | 질문 내용 |
| create_date | datetime | NOT NULL | 작성일 |
| modify_date | datetime | NULL | 수정일 |
| author_id | integer | NOT NULL | 작성자 ID |
| FOREIGN KEY (author_id) REFERENCES auth_user(id) | | | |

테이블 이름: posts_question_voter

| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 고유 ID |
| question_id | bigint | NOT NULL | 질문 ID |
| user_id | integer | NOT NULL | 사용자 ID |
| FOREIGN KEY (question_id) REFERENCES posts_question(id) | | | |
| FOREIGN KEY (user_id) REFERENCES auth_user(id) | | | |

# | 테이블 정의서 - ERD



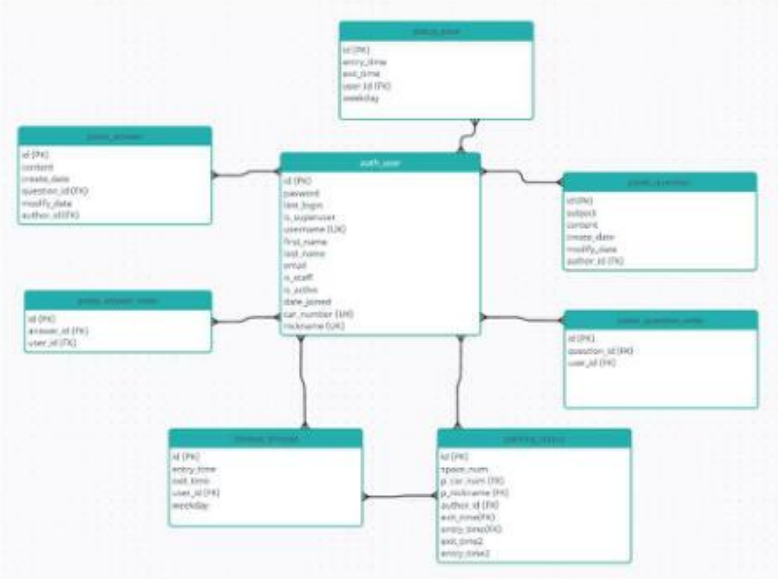
테이블 이름: timeset_timeset

| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 고유 ID |
| entry_time | time | NOT NULL | 입차 시간 |
| exit_time | time | NOT NULL | 출차 시간 |
| user_id | integer | NOT NULL | 사용자 ID |
| weekday | varchar(10) | NOT NULL | 요일 |
| FOREIGN KEY (user_id) REFERENCES auth_user(id) | | | |

테이블 이름: status_save

| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 고유 ID |
| entry_time | time | NOT NULL | 입차 시간 |
| exit_time | time | NOT NULL | 출차 시간 |
| user_id | integer | NOT NULL | 사용자 ID |
| weekday | varchar(10) | NOT NULL | 요일 |
| FOREIGN KEY (user_id) REFERENCES auth_user(id) | | | |

# | 테이블 정의서 - ERD



| 컬럼 이름 | 데이터 타입 | 제약 조건 | 설명 |
|---|---|---|---|
| id | integer | PRIMARY KEY, AUTOINCREMENT | 고유 ID |
| plate_text | varchar(20) | NOT NULL | 차량 번호판 텍스트 |
| index | integer | NOT NULL | 주차 공간 인덱스 |
| entry_time | datetime | NOT NULL | 입차 시간 |
| exit_time | datetime | NULL | 출차 시간 |

추가 데이터베이스 테이블
[ Parking history ]

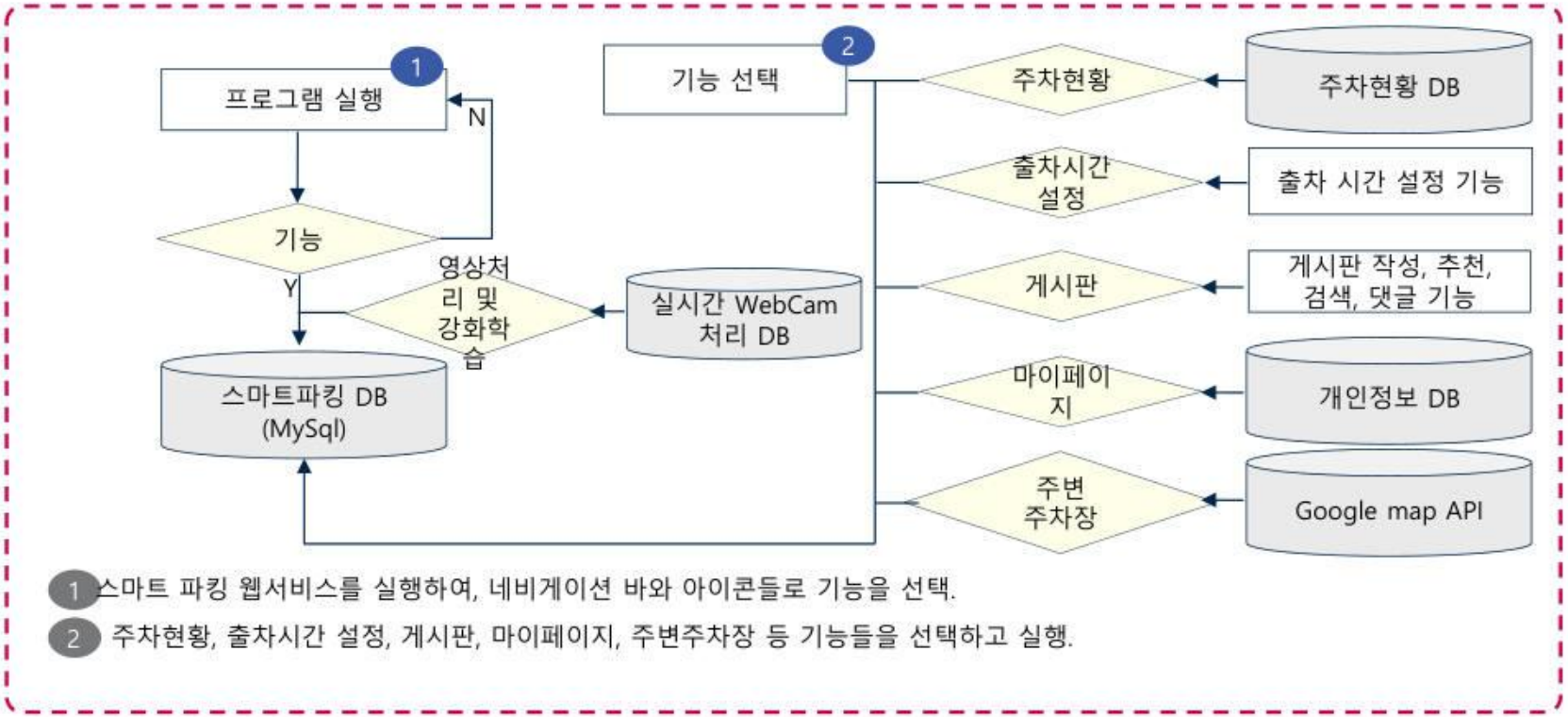
# | 기능 처리도(기능 흐름도)

실무 산출물 형식

| 프로그램 ID | SMT-P01 | 프로그램 명 | 스마트 파킹 | 작성일 | 2024 . 07 . 09 . | Page | 1/1 |
|---|---|---|---|---|---|---|---|
| 개요 | 실시간 캠으로 받아온 주차 현황 영상을 처리하고 머신러닝으로 강화학습을 진행하여 주차현황과 차량들의 출차 예상시간을 웹으로 제공. | | | | | 작성자 | 팀장 |

## 기능 흐름도

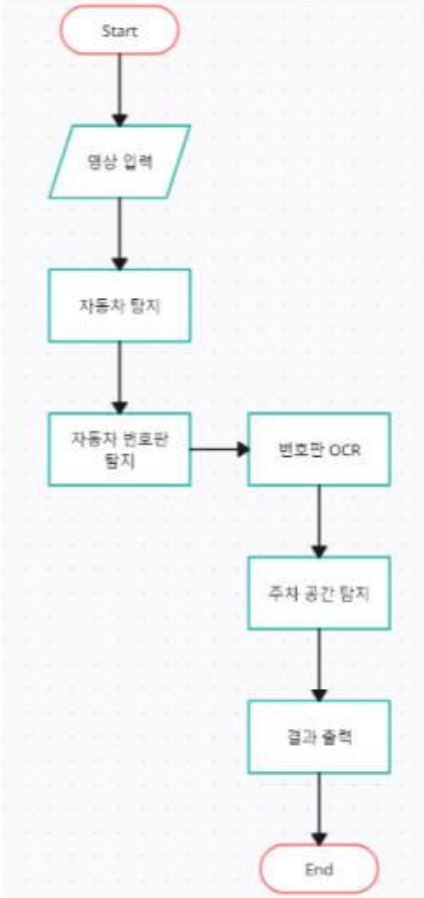
<키넥트 연동 인터렉티브 스마트파킹 연계도>




1 스마트 파킹 웹서비스를 실행하여, 네비게이션 바와 아이콘들로 기능을 선택.

2 주차현황, 출차시간 설정, 게시판, 마이페이지, 주변주차장 등 기능들을 선택하고 실행.

# | 알고리즘 명세서




## 시나리오

**1.Start**: 시스템이 시작되면, 카메라 또는 비디오 파일에서 영상을 입력 받음.
**2.영상 입력**: 입력된 영상을 프레임 단위로 분할.
**3.자동차 탐지**: YOLO, Faster R-CNN 등의 객체 탐지 알고리즘을 사용하여 프레임 내에서 자동차를 탐지.
**4.자동차 번호판 탐지**: 탐지된 자동차 영역에서 번호판을 추가로 탐지.
**5.번호판 OCR**: 탐지된 번호판 영역에서 Tesseract OCR을 사용하여 번호판의 문자를 인식.
**6.주차 공간 탐지**: 주차 공간 내 빈 공간을 탐지하고 현재 상태를 파악.
**7.결과 출력**: 탐지된 결과(자동차 위치, 번호판 정보, 주차 공간 상태)를 출력.
**8.End**: 모든 처리가 완료된 후 시스템을 종료.

## 빈주차공간탐지 사진 변경, +OCR 보정 내용 설명 추가

### 1. 객체 탐지 알고리즘
### 자동차 및 자동차 번호판 OCR, 주차 공간 탐지

객체 탐지(Object Detection) : 영상 내 특정 객체를 인식하고 위치를 추적하는 기술.
이 기술을 자동차와 자동차의 번호판, 그리고 주차 공간을 탐지하는 데 활용할 수 있음.

- **자동차 탐지**: YOLO(You Only Look Once), Faster R-CNN과 같은 딥러닝 기반 모델을 사용하여 영상 내 자동차를 탐지. 이 모델들은 높은 정확도와 빠른 속도를 제공하며, 다양한 크기와 형태의 자동차를 인식할 수 있음.
- **자동차 번호판 OCR**: 번호판 인식에는 먼저 번호판을 탐지한 후, 그 영역을 잘라 OCR(Optical Character Recognition) 알고리즘을 적용하여 번호판의 문자를 인식. Tesseract OCR과 같은 오픈소스 라이브러리를 활용할 수 있으며, 이를 통해 번호판의 텍스트를 추출.
- **주차 공간 탐지**: 주차 공간을 탐지하기 위해서는 주차장의 영역을 정의하고, 해당 영역 내의 빈 공간을 식별하는 알고리즘을 사용. 주로 영상 처리 기술을 사용하여 주차 공간의 상태(빈 공간, 주차된 차량)를 실시간으로 모니터링.



<객체 탐지>

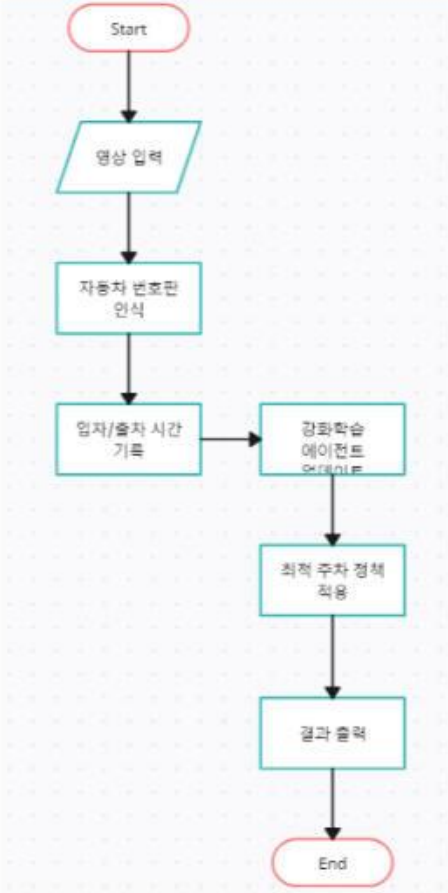


<빈 주차 공간 탐지>




<번호판 OCR>

# | 알고리즘 명세서




## 시나리오

**1.Start**: 시스템이 시작되면, 카메라 또는 비디오 파일에서 영상을 입력 받음.
**2.영상 입력**: 입력된 영상을 프레임 단위로 분할.
**3.자동차 번호판 인식**: 번호판 OCR을 통해 차량의 번호판을 인식.
**4.입차/출차 시간 기록**: 인식된 번호판 정보를 기반으로 차량의 입차 및 출차 시간을 데이터베이스에 기록.
**5.강화학습 에이전트 업데이트**: 기록된 데이터를 기반으로 강화학습 에이전트를 업데이트하여 주차 공간의 효율적 배치를 학습.
**6.최적 주차 정책 적용**: 학습된 정책을 기반으로 최적의 주차 공간을 추천하고 배정.
**7.결과 출력**: 최적 주차 공간 배정 결과를 출력.
**8.End**: 모든 처리가 완료된 후 시스템을 종료.

# 강화학습에서 정책학습과 보상함수 대신에 가중치 알고리즘 설명 적기 (자세하게)

## 2. 차량의 출차 입차 시간 기록 및 강화학습 알고리즘

### 출차 입차 시간 기록

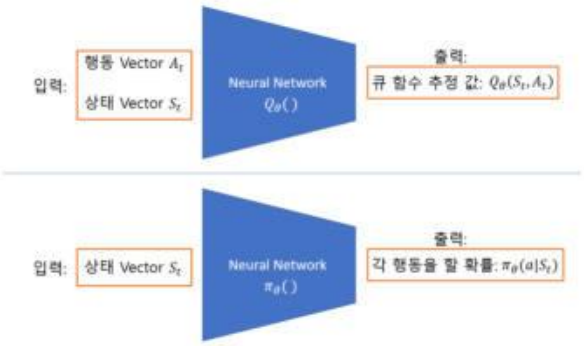
차량의 출차 및 입차 시간을 기록하는 시스템은 주차장의 관리와 운영에 있어 중요한 역할.

- **시스템 구현**: 차량이 주차장에 입차하거나 출차할 때 번호판을 인식하여 차량의 입차 및 출차 시간을 자동으로 기록. 이 기록은 데이터베이스에 저장되며, 추후 주차 요금 계산 및 주차 공간의 효율적 관리에 활용됨.

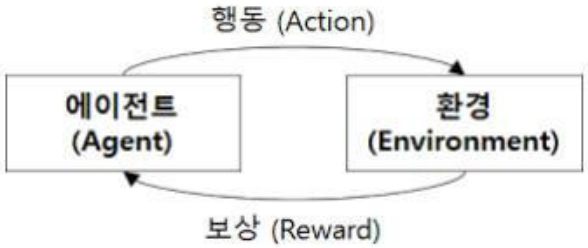
### 강화학습 알고리즘

강화학습(Reinforcement Learning) : 에이전트가 환경과 상호작용하며 보상을 최대화하는 행동을 학습하는 방법.
차량의 출차 및 입차 시간을 기반으로 주차장 관리 시스템에 적용할 수 있는 강화학습 알고리즘은 다음과 같음.

- **정책 학습**: 주차 공간의 효율적 배치를 위해 강화학습을 적용. 에이전트는 각 주차 공간의 상태(빈 공간, 주차된 차량)와 차량의 입차 및 출차 시간을 고려하여 최적의 주차 정책을 학습. (가중치 그래프 사용)
- **보상 함수**: 주차 공간의 활용도를 극대화하고, 입차 및 출차 시간의 효율성을 높이기 위해 보상 함수를 설계. ex) 주차 공간의 효율적 사용과 차량의 빠른 입출차를 보상으로 설정.

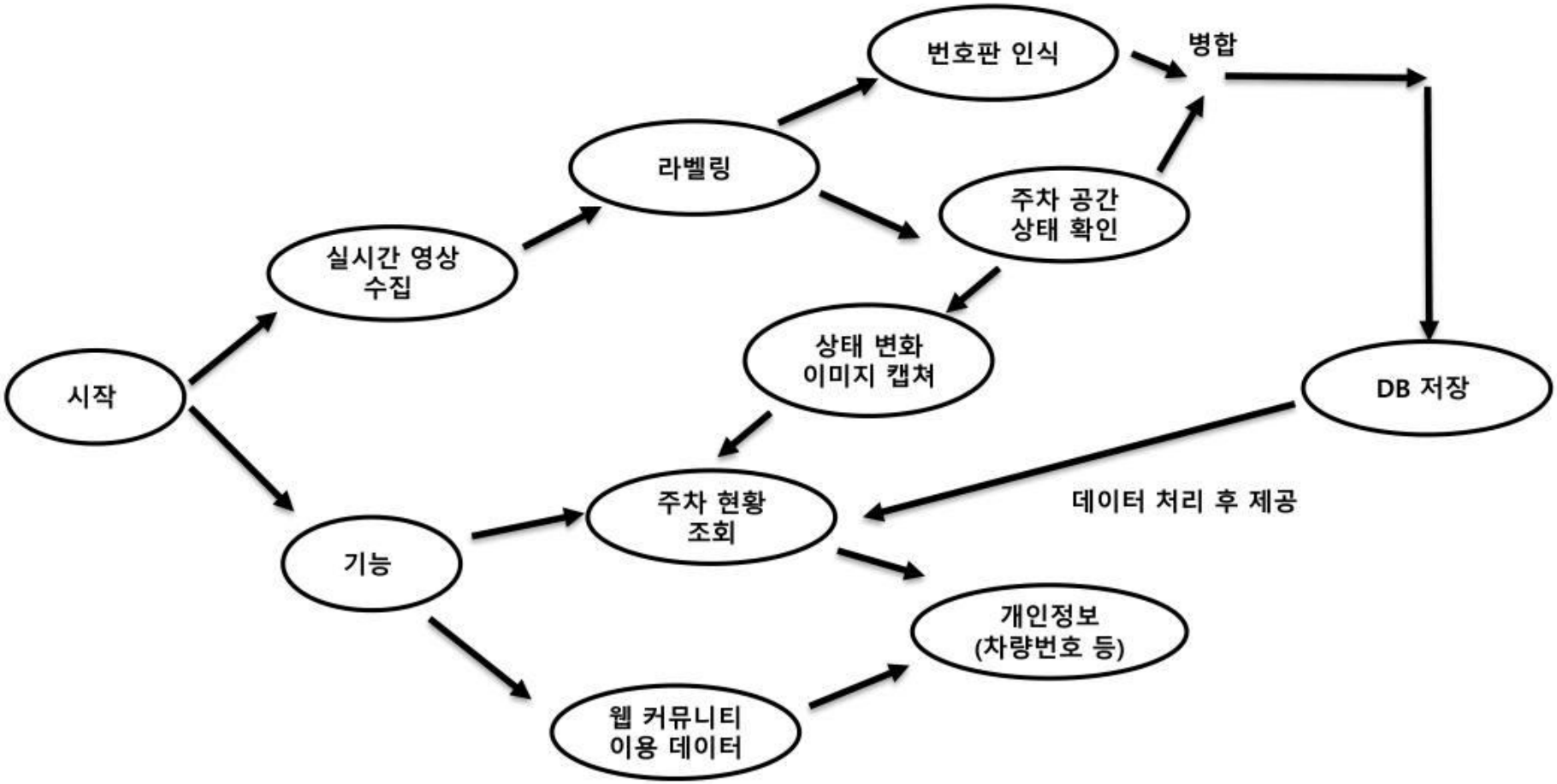



<정책 학습>




<보상 함수>

# | 데이터 수집처리 정의서


번호판 인식
병합
라벨링
주차 공간
상태 확인
실시간 영상
수집
상태 변화
이미지 캡쳐
시작
DB 저장
주차 현황
조회
데이터 처리 후 제공
기능
개인정보
(차량번호 등)
웹 커뮤니티
이용 데이터

# | 프로그램 - 목록

| 기능 분류 | 기능번호 | 기능 명 |
|---|---|---|
| LOG | LOG-01-01 | 로그인 전 화면 |
| | LOG-01-02 | 로그인 |
| | LOG-01-03 | 아이디/비밀번호 찾기 |
| | LOG-01-04 | 회원가입 |
| HOME | HOME-01-01 | 홈 화면 |
| | HOME-01-02 | 주차 현황 페이지 |
| | HOME-01-03 | 출차 시간 설정 페이지 |
| | HOME-01-04 | 게시판 목록 페이지 |
| | HOME-01-04-01 | 자유게시판 페이지 |
| | HOME-01-04-02 | 공지사항 페이지 |
| | HOME-01-04-03 | 오픈채팅 초대 페이지 |
| MY | MY-01 | 프로필 설정 |
| 이하 생략 | | |

## | 핵심소스코드(1)

### 데이터 학습

```
!pip install roboflow

from.roboflow import Roboflow
rf = Roboflow(api_key="")
project = rf.workspace("kangdo-6svxj").project("parking-rphnx")
version = project.version(2)
dataset = version.download ("yolov8")
```

```
from ultralytics import YOLO

# YOLOV8 모델 불러오기 (예시: yolov8n.pt)
model = YOLO('yolov8n.pt')

#데이터셋 경로 설정
data_path = '/content/Parking-2/data.yaml'

#모델 훈련
model.train(data=data_path, epochs=100, imgsz=640)
```

```
from ultralytics import YOLO
from IPython.display import Image, display
import os

# YOLO 모델 불러오기
model = YOLO('/content/runs/detect/train/weights/best.pt')

# 이미지에서 예측 수행 및 결과 저장
results = model.predict(source='/content/Parking-2/train/images',
save=True, project='/content/runs', name='predict_results')

# 예측 결과가 저장된 디렉토리 경로
result_dir = '/content/runs/predict_results'

# 결과 디렉토리 내의 파일 목록 출력 및 시각화
for file_name in os.listdir(result_dir):
    if file_name.endswith(('.png', '.jpg', '.jpeg')):
        display(Image(filename=os.path.join(result_dir,file_name)))
```

## | 핵심소스코드(2)

1. 웹캠을 통해 실시간 영상을 받아오는 처리

```python
import cv2

def capture_video():
    cap = cv2.VideoCapture(1)  # 웹캠 장치 번호는 1번
    if not cap.isOpened():
        print("Error: Could not open webcam.")
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Error: Could not read frame.")
            break

        cv2.imshow("Webcam", frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

capture_video()
```

2. 5분마다 실시간 영상을 캡쳐 후 이미지로 저장

```python
import cv2
import time
from datetime import datetime

def capture_images_every_5_minutes():
    cap = cv2.VideoCapture(0)
    if not cap.isOpened():
        print("Error: Could not open webcam.")
        return

    while True:
        ret, frame = cap.read()
        if not ret:
            print("Error: Could not read frame.")
            break

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filename = f"webcam_capture_{timestamp}.jpg"
        cv2.imwrite(filename, frame)
        print(f"Saved image: {filename}")

        time.sleep(300)  # 5분 대기

    cap.release()

capture_images_every_5_minutes()
```

# | 핵심소스코드(3)

## 3. Tesseract OCR로 번호판 처리

```python
import cv2
import pytesseract

def process_license_plate(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

    text = pytesseract.image_to_string(binary, config='--psm 8')
    print(f"Detected license plate: {text}")

image_path = "path_to_captured_image.jpg"
process_license_plate(image_path)
```

## 4. 주차 공간 상태 확인 및 라벨링

```python
import cv2
import numpy as np

def detect_parking_spaces(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(binary, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

    parking_spaces = {}
    index = 1
    for contour in contours:
        area = cv2.contourArea(contour)
        if area > 500:  # 주차 공간 크기 기준
            x, y, w, h = cv2.boundingRect(contour)
            roi = img[y:y+h, x:x+w]
            space_status = "empty" if is_empty(roi) else "full"
            parking_spaces[index] = space_status
            index += 1

    print(parking_spaces)

def is_empty(roi):
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)
    white_pixels = np.sum(binary == 255)
    return white_pixels > (roi.size / 2)

image_path = "path_to_captured_image.jpg"
detect_parking_spaces(image_path)
```

# | 핵심소스코드(4)

5. 주차 공간과 번호판 라벨링 결합 및 DB 저장

```
import cv2
import pytesseract
import django
from django.conf import settings
from django.db import models

# Django 설정
settings.configure(
    DATABASES={
        'default': {
            'ENGINE':
'django.db.backends.mysql',
            'NAME': 'your_db_name',
            'USER': 'your_db_user',
            'PASSWORD': 'your_db_password',
            'HOST': 'localhost',
            'PORT': '3306',
        }
    }
)
django.setup()
```

```
class ParkingStatus(models.Model):
    space_num = models.IntegerField()
    status = models.CharField(max_length=10)
    license_plate = models.CharField(max_length=20)

def process_and_save_to_db(image_path):
    img = cv2.imread(image_path)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)

    contours, _ = cv2.findContours(binary, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)

    index = 1
    for contour in contours:
        area = cv2.contourArea(contour)
        if area > 500:  # 주차 공간 크기 기준
            x, y, w, h = cv2.boundingRect(contour)
            roi = img[y:y+h, x:x+w]
            space_status = "empty" if is_empty(roi) else "full"
            license_plate = pytesseract.image_to_string(roi, config='--psm 8').strip()

            # DB에 저장
            parking_status = ParkingStatus(space_num=index, status=space_status,
license_plate=license_plate)
            parking_status.save()
            index += 1

def is_empty(roi):
    gray = cv2.cvtColor(roi, cv2.COLOR_BGR2GRAY)
    _, binary = cv2.threshold(gray, 128, 255, cv2.THRESH_BINARY)
    white_pixels = np.sum(binary == 255)
    return white_pixels > (roi.size / 2)

image_path = "path_to_captured_image.jpg"
process_and_save_to_db(image_path)
```

# | 핵심소스코드(5)

Django 모델 설정

```python
# models.py

from django.db import models

class ParkingStatus(models.Model):
    space_num = models.IntegerField()
    status = models.CharField(max_length=10)
    license_plate = models.CharField(max_length=20)
    timestamp = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"Space {self.space_num}: {self.status}
({self.license_plate})"
```

Django 설정 파일 수정

```python
# settings.py


DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'your_db_name',
        'USER': 'your_db_user',
        'PASSWORD': 'your_db_password',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

# | 핵심소스코드(6)

## 주차 상태 데이터 함수

```
def parking_status(request):
    # 주차 상태 데이터 가져오기
    parking_spaces = ParkingSpace.objects.all()

    # 사용자별로 추천된 주차 공간 가져오기
    user = request.user
    if user.is_authenticated:
        car_number = user.car_number
        recommended_space = recommend_parking_space(car_number)
    else:
        recommended_space = None

    context = {
        'parking_spaces': parking_spaces,
        'recommended_space': recommended_space,  # 추천된 주차 공간 추가
    }

    return render(request, 'parking_status/parking_status.html', context)
```

## 요일별 평균 입차 및 출차 시간 계산

```
def calculate_average_times():
    """요일별 평균 입차 시간, 출차 시간, 머무르는 시간 계산"""
    days_of_week = ['월요일', '화요일', '수요일', '목요일', '금요일', '토요일', '일요일']
    average_data = {}

    for i in range(7):
        histories = ParkingHistory.objects.filter(entry_time__week_day=i + 1)  # 1: 일요일, ..., 7: 토요
일


        if histories.exists():
            total_entry_time = timedelta()
            total_exit_time = timedelta()
            total_duration = timedelta()
            count = histories.count()

            for history in histories:
                entry_time_of_day = history.entry_time.time()
                exit_time_of_day = history.exit_time.time()
                duration = history.exit_time - history.entry_time

                total_entry_time += timedelta(hours=entry_time_of_day.hour,
minutes=entry_time_of_day.minute)
                total_exit_time += timedelta(hours=exit_time_of_day.hour,
minutes=exit_time_of_day.minute)
                total_duration += duration

            average_entry_time = (total_entry_time / count).total_seconds()
            average_exit_time = (total_exit_time / count).total_seconds()
            average_duration = (total_duration / count).total_seconds()

            average_data[days_of_week[i]] = {
                'average_entry_time': str(timedelta(seconds=average_entry_time)),
                'average_exit_time': str(timedelta(seconds=average_exit_time)),
                'average_duration': str(timedelta(seconds=average_duration))
            }

    return average_data
```

# | 핵심소스코드(7)

## 예상 출차 시간 계산

```
def calculate_estimated_exit_time(plate_text):
    # 예시 알고리즘: 과거의 평균 주차 시간을 사용하여 예상 출차 시간을 계산
    history = ParkingHistory.objects.filter(plate_text=plate_text).order_by('-entry_time').first()
    if history:
        average_duration = ParkingHistory.objects.filter(plate_text=plate_text).aggregate(avg_duration=Avg(F('exit_time') - F('entry_time')))['avg_duration']
        if average_duration:
            return history.entry_time + average_duration
    return None
```

## 추천 주차 자리 계산

```
def recommend_parking_space(plate_text):
    # 특정 차량의 과거 주차 패턴 분석
    histories = ParkingHistory.objects.filter(plate_text=plate_text)

    # 주차 공간별로 점수를 매기기 위한 딕셔너리
    score_dict = defaultdict(int)

    for history in histories:
        # 주차된 인덱스에 대해 점수 부여
        score_dict[history.index] += 1

        # 주차된 요일에 대해 가중치 부여
        weekday = history.entry_time.weekday()
        score_dict[history.index] += weekday

        # 주차 시간의 패턴에 따라 가중치 부여 (예: 출퇴근 시간대)
        entry_hour = history.entry_time.hour
        if 7 <= entry_hour <= 9 or 17 <= entry_hour <= 19:
            score_dict[history.index] += 2

    # 모든 주차 공간의 리스트 가져오기
    all_spaces = ParkingSpace.objects.all()

    # 주차공간이 비어있지 않은 경우에도 점수를 고려하여 추천
    recommended_space = None
    max_score = -1

    for space in all_spaces:
        # 점유되지 않은 주차공간만 고려
        if not space.is_occupied:
            space_score = score_dict.get(space.index, 0)
            if space_score > max_score:
                recommended_space = space
                max_score = space_score

    # 만약 최적의 주차 공간을 찾지 못한 경우, 첫 번째 빈 주차 공간을 반환
    if recommended_space is None:
        recommended_space = all_spaces.filter(is_occupied=False).first()

    return recommended_space
```

# | 참조 - S/W 기능 실사사진


주차의 달인
Login Details
로그인을 위해 아이디와 비밀번호를 입력하세요.
ID
admin
Password
회원가입
로그인
© 2024 한이음 빌라 | 전화: 010-xxxx-xxxx | 이메일: admin@example.com


주차의 달인
한이음 빌라
이용 현황
현재 주차장 이용 현황을 확인해보세요.
자리가 부족할 경우 대체 주차장을 이용해보세요.


주차의 달인
주변 주차장
지도
위성
Google
© 2024 한이음 빌라 | 전화: 010-xxxx-xxxx | 이메일: admin@example.com

# | 참조 - H/W 기능 실사사진






## | 참조 - H/W 기능 실사사진


Full 0.94
car 0.89
car 0.79
car 0.83
plate 0.27
34나 5678
plate 0.42
car 0.81
car 0.82
plate 0.54
63소 4478
plate 0.49
57누 1267
290아 4830

# | 참조 - 개발 환경 및 설명

| 구분 | | 항목 | 적용내역 |
|---|---|---|---|
| S/W 개발환경 | 웹 개발 | Djnago 2024.1.4 | 웹 서비스 백엔드 개발 |
| | | Html, css | 웹 프론트엔드 개발 |
| | | Windows 11 OS | 컴퓨터 운영체제 |
| | 서버 애플리케이션 개발 | Sqlite(5.1.73) | 유저 및 관리자, 주차현황, 게시판 등 웹 서비스와 관련된 데이터를 저장, 관리하는 데이터베이스 |
| | | Apache(1.7.1) | 관리자 웹 페이지를 구동하는 웹 서버 |
| | | 서버 운영체제 | 리눅스 CentOS 6.6 |
| | 영상 처리 | YOLO | 영상처리 인식(OCR 및 객체 인식) |
| | 딥러닝 학습 | Google colab | YOLO 처리를 위한 라벨링 데이터 딥러닝을 위해 GPU 서버 사용 |
| | | Roboflow | YOLO 처리를 위한 데이터 라벨링 처리 |
| H/W 구성장비 | 디바이스 | Web cam(UHD2160) | PC에 영상처리할 주차 현황 영상을 실시간 제공 |
| | | PC | 소프트웨어 코드 처리 및 웹캠과 연결하여 영상 처리 및 서버에 JSON으로 송신 및 업로드 |
| | 통신 | USB 어댑터 | WebCam과 컴퓨터의 연결 |
| | | 랜 어댑터 | 웹 서버 통신 |
| | 서버 | 클라우드 서버 | SW 개발 및 테스트를 위한 클라우드 서버 (Microsoft Azure Server) |

# Thank you

